

Software Architecture and Design Specification

Project: Task Manager Application

Version: 1.0

Authors: Team 4 — OM SATYAJYOTI PATTANAYAK; PRACHI GANESH JOSHI; PRIYANSH SONI; RAJAT KIRAN KOOGE

Date: 02-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 02-10-2026 | Team 4 | Initial architecture and design specification

Approvals

Role | Name | Signature/Date

Team Members | Team 4 | Pending

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Task Manager Application.

1.2 Scope

Covers user registration, authentication, task creation, task viewing, editing, deletion, assignment, status and priority management, search/filtering, and dashboard-based progress tracking.

1.3 Audience

Developers, QA engineers, project instructors, security reviewers, and maintenance teams.

1.4 Definitions

SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), API (Application Programming Interface), JWT (JSON Web Token), REST (Representational State Transfer).

2. Document Overview

2.1 How to use this document

Provides the architecture and design specifications for the Task Manager Application, including the component architecture, architecture pattern, requirement traceability, security architecture, UML sequence diagrams, API design, error handling, and related design decisions.

2.2 Related Documents

SRS, Software Test Plan (STP), Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: secure, maintainable, reliable, usable task management with clear separation of concerns. **Constraints:** React.js frontend, Node.js backend, web-based deployment, database persistence, HTTP/HTTPS communication, and the SRS requirements defined for the project.

3.2 Stakeholders & Concerns

Users: usability, task visibility, and reliable task updates

Team members: assignment, status tracking, and collaboration

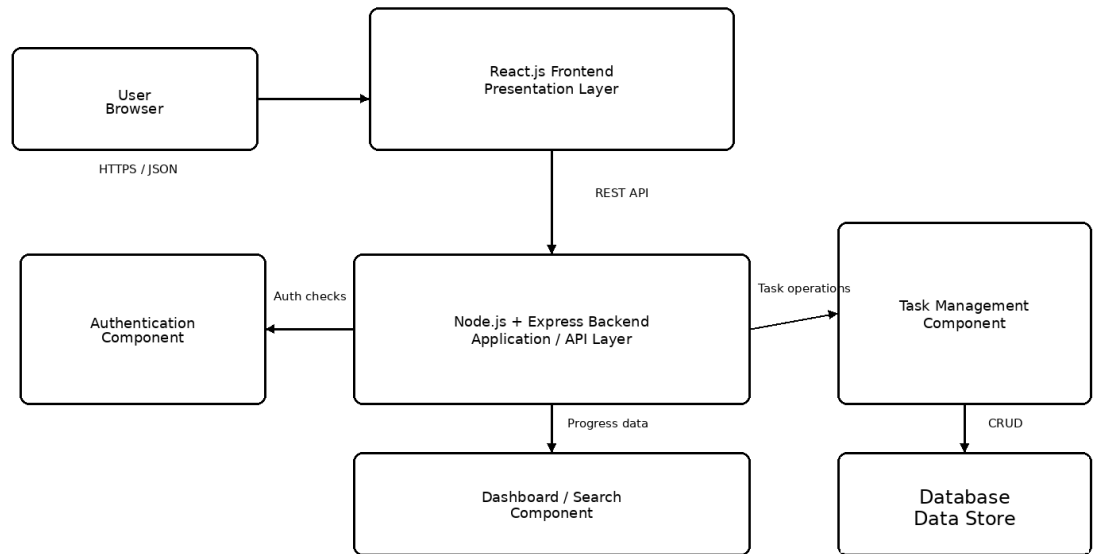
Developers: modularity and maintainability

QA engineers: testability and traceability

Project instructors: conformance to software engineering deliverables and requirements.

3.3 Component (UML) Diagram

Task Manager Application — UML Component Diagram



3.4 Component Descriptions

- React Frontend: provides the user interface for registration, login, task management, search/filtering, and dashboard views.
- Authentication Module: validates credentials and manages authenticated access.
- Task Management Module: creates, retrieves, updates, assigns, and deletes tasks and manages task status and priority.
- Dashboard / Search Module: prepares task progress information for presentation to users.
- Node.js/Express API: exposes REST endpoints, validates requests, applies authorization checks, and coordinates backend services.
- Database: stores user, task, assignment, status, priority, and related application data.

3.5 Chosen Architecture Pattern and Rationale

A layered three-tier architecture is chosen to separate presentation, application/business logic, and data management. This supports modular development, maintainability, testability, and clear responsibility boundaries. A microservices architecture is not selected because the current project scope can be implemented more simply as a modular single backend.

3.6 Technology Stack & Data Stores

Frontend: React.js. Backend: Node.js with Express.js. Communication: REST-style HTTP/HTTPS with JSON. Data store: database configured for persistent user and task data. Development and testing tools include a modern web browser, Node.js tooling, Git, and API testing tools.

3.7 Risks & Mitigations

Risk: unauthorized access to protected task operations → Mitigation: authentication and authorization checks for protected API endpoints.

Risk: invalid task data → Mitigation: server-side input validation.

Risk: backend or database failure → Mitigation: standardized error handling, validation, and reliable persistence practices.

Risk: changes in one module affecting others → Mitigation: layered modular design and defined API interfaces.

3.8 Traceability to Requirements

FR-01, FR-02 → Authentication Module

FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-09, FR-10 → Task Management Module / Node.js API

FR-11 → Dashboard Module

NFR-01 → Node.js API and database interaction

NFR-02 → Authentication and authorization controls

NFR-03 → React Frontend

NFR-04 → Database and backend persistence

NFR-05, NFR-06 → Layered modular architecture

SR-01, SR-02 → Authentication and authorization controls

SR-03 → Backend security and logging practices

SR-04 → API input validation.

3.9 Security Architecture

Security architecture follows defense-in-depth principles. Authentication is required before protected operations. Authorization checks are applied before users access or modify task resources. API input is validated before processing. Passwords are not exposed in responses or normal logs. HTTPS is used for secure communication in deployment. Access to task data is restricted according to the authenticated user and authorized task relationships.

4. Design

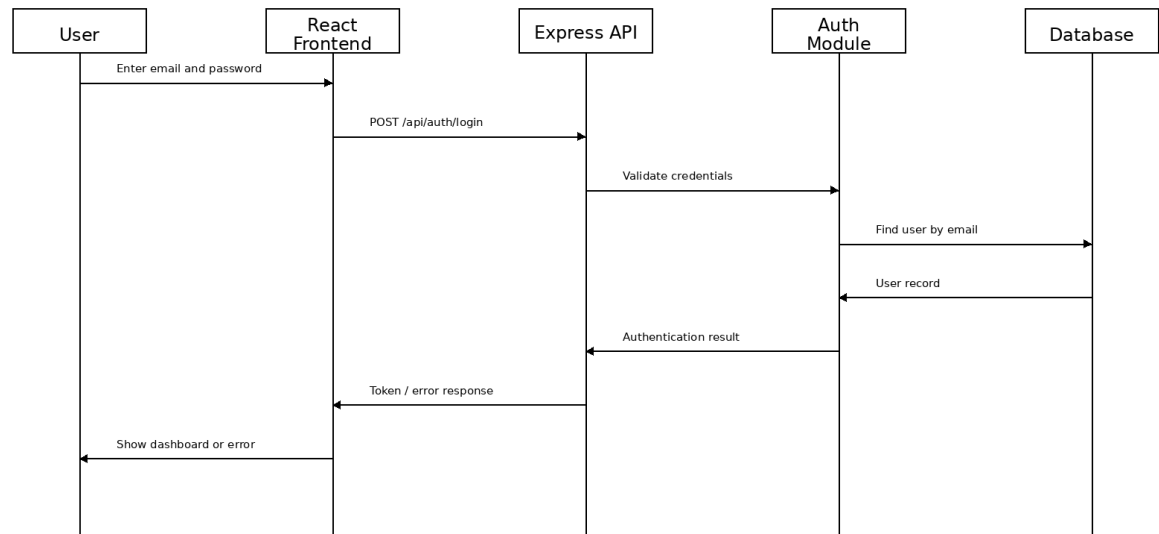
4.1 Design Overview

The Task Manager Application is designed using a three-tier layered structure: the React presentation layer communicates with the Node.js/Express application layer through REST APIs, while the application layer manages authentication, authorization, task operations, validation, and dashboard data access. The database layer provides persistent storage.

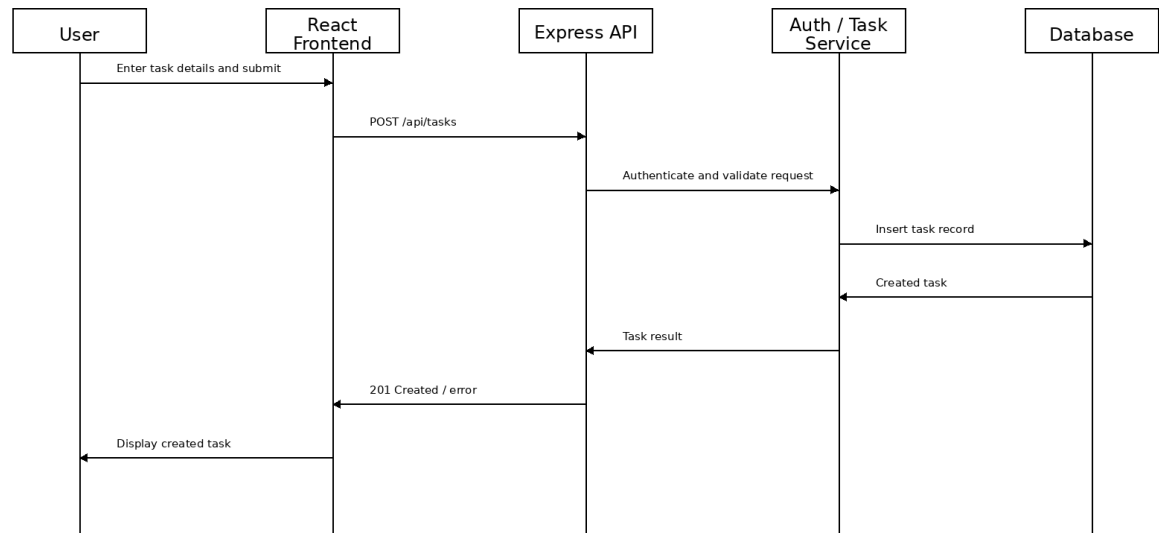
4.2 UML Sequence Diagrams

The following sequence diagrams cover two primary project flows: user authentication and task creation.

Task Manager — UML Sequence Diagram: User Login



Task Manager — UML Sequence Diagram: Create Task



4.3 API Design

The API design defines interfaces between the React frontend and Node.js/Express backend. Authentication and task management are the primary API components.

Authentication API

POST /api/auth/register

Request: { email, password }

Response: { message, user }

POST /api/auth/login

Request: { email, password }

Response: { token, user }

Errors: 400 Invalid input, 401 Invalid credentials, 409 Email already registered.

Task API

GET /api/tasks

POST /api/tasks

PUT /api/tasks/:id

DELETE /api/tasks/:id

PUT /api/tasks/:id/status

PUT /api/tasks/:id/assign

Responses use JSON. Protected task endpoints require authenticated access.

Errors: 400 Invalid request, 401 Authentication required, 403 Unauthorized access, 404

Task not found, 500 Internal server error.

4.4 Error Handling, Logging & Monitoring

Standardized HTTP status codes and JSON error responses are used. Client-side errors are returned with clear messages without exposing sensitive implementation details.

Authentication credentials and other sensitive information are not written to normal application logs. Backend monitoring can track request failures, authentication failures, and server/database errors during deployment.

4.5 UX Design

The React interface should provide consistent navigation and forms for registration, login, task creation, task editing, task assignment, status updates, search/filtering, and dashboard viewing. Validation and clear error messages should be presented near the relevant user action.

4.6 Open Issues & Next Steps

Future work may include notifications, advanced filtering and sorting, richer analytics, role-based access enhancements, activity history, and additional collaboration features, provided they remain within the approved project scope.

5. Appendices

5.1 Glossary: SRS, SAD, STP, RTM, API, JWT, REST, React.js, Node.js, Express.js.

5.2 References: Software Requirements Specification for the Task Manager Application; Software Engineering Mini-Project Deliverables Part-1; supplied SAD template.

5.3 Tools: draw.io, PlantUML or equivalent UML tools, React.js, Node.js/Express.js, Git, API testing tools.